20. Arquitetura Geral do Sistema

O MarIA é um HUB de continuidade do atendimento centrado no médico, com identidade central do paciente e compartilhamento controlado por autorização pessoal. A clínica identifica vínculos e procedência, sem delimitar um tenant no novo portal.

A implementação da Fase 02 utiliza banco e schema compartilhados. Não cria schemas exclusivos por profissional ou paciente e não utiliza django-tenant-schemas. O acesso é delimitado no backend por perfil, autoria, vínculo e Grant.

20.1 Visão Geral dos Componentes e Camadas

1. Camada de Apresentação (Portal Web)

• Médico: carteira, perfil, autorizações, histórico, atendimentos e visualizador de documentos.

• Paciente: histórico pessoal, documentos, perfil, identificador e decisão das autorizações.

• HTML, CSS nativo e JavaScript reaproveitam o frontend anterior. Não há Tailwind nem aplicativo móvel em execução.

2. Camada de Negócio e Orquestração

Django e DRF recebem HTTP e JSON. Views e serializers tratam o contrato; permissions verificam identidade e contexto; services aplicam regras transacionais; selectors centralizam consultas. JWT autentica as APIs.

Agenda, caixa, filas Celery/Redis e emissão de documentos assinados não integram o incremento atual.

3. Camada de Inteligência Artificial Assistiva

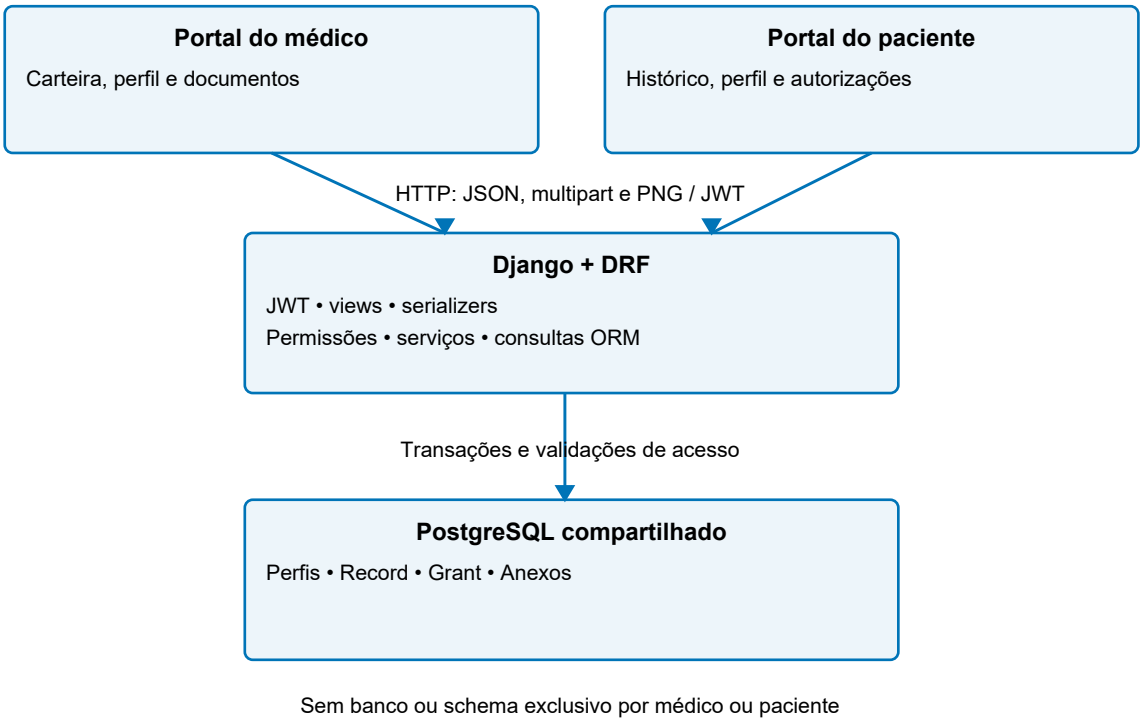
Permanece planejada. Não há captura de áudio, transcrição ou serviço de IA em execução. Eventual adoção exige avaliação técnica e de privacidade; não se declara certificação ISO/IEC 42001.

4. Camada de Dados, Persistência e Segurança

PostgreSQL compartilhado com ORM, migrações e restrições. Patient centraliza a identidade; DoctorProfile habilita o profissional; Record preserva autor e origem; Grant delimita compartilhamento; Audit registra operações.

Registros próprios são consultados em rota separada do histórico compartilhado. A revogação não exclui dados. O ambiente é local, com HTTP em localhost e dados fictícios; proteção de produção ainda exige revisão.

20.2 Diagrama Lógico do Fluxo da Arquitetura



Sem banco ou schema exclusivo por médico ou paciente

21 TECNOLOGIAS E FERRAMENTAS DO PROJETO

21.1 Tabela Resumo da Stack Tecnológica

Ferramenta / Linguagem	Camada de Atuação	Justificativa e Aplicação no Sistema
Python 3.12	Backend	Linguagem das regras de negócio e serviços.
Django 5.2.18 / DRF 3.16.1	Backend / API	ORM, templates, autenticação, validação, rotas e transações.
PostgreSQL / psycopg	Dados	Persistência relacional compartilhada; sem schemas individuais.
HTML5 / CSS3 / JavaScript	Frontend	Base visual reaproveitada e comunicação Fetch com a API.
SimpleJWT 5.5.1 / PyJWT	Autenticação	Tokens existentes para login e chamadas protegidas. Revisão da chave local pendente.
Docker Compose	Ambiente	PostgreSQL local em contêiner, com volume persistente.
Figma / Git / GitHub	Design e versionamento	Referência visual e repositório da equipe. Incremento local ainda não enviado.
DBeaver / testes Django	Inspeção e qualidade	Visualização do modelo e validação automatizada no PostgreSQL.

Não implementados: Tailwind, Celery, Redis, django-tenant-schemas, aplicativo móvel, IA e assinatura ICP-Brasil. Não se afirma TLS de produção, AES-256 em repouso ou certificação normativa.

21.2 Detalhamento e Justificativa das Tecnologias Adotadas

1. Camada de Backend e Orquestração

Python/Django/DRF permitem tratar entrada HTTP, regras e persistência com responsabilidades separadas. O domínio implementado está em backend/hub; configurações e rotas em backend/config. A aplicação não possui os pacotes /core e /apps descritos na proposta anterior.

2. Camada de Dados e Controle de Acesso

O banco PostgreSQL utiliza o schema public compartilhado. O filtro por autoria e a validação de Grant delimitam a leitura. Integridade estrutural é tratada por chaves estrangeiras, unicidade, checks e transações. Não há middleware de seleção de tenant ou RLS.

Migrações 0003 e 0004 preservaram o banco e a seed existentes, criaram perfis médicos e permitiram atendimentos diretos. Grants antigos de clínica não se convertem em consentimento pessoal.

3. Camada de Apresentação e UX

O portal reutiliza a identidade azul, imagens e estrutura do frontend manual. Templates Django e API operam na mesma origem. JavaScript monta a interface a partir de dados reais, com mensagens de erro e controle do envio de formulários.

4. Segurança e Evolução

JWT autentica as chamadas; o backend autoriza cada operação. Segredos ficam no .env local. HTTPS, proteção de repouso, política de retenção, verificação profissional real e revisão das chaves são pendências para produção. Não há afirmação de conformidade jurídica automática.

22. Implementação por Camadas do Sistema

As camadas implementadas abrangem apresentação web, API e persistência compartilhada. A Fase 02 demonstra comunicação real em fluxos completos. Agenda, finanças, emissão certificada e IA continuam planejadas. Anexos privados e perfis estão integrados.

22.1 Implementação Frontend

• Login real em /hub/login/; identificação do perfil por GET /api/v1/me/.

• Carteira e busca por nome; criação e listagem de solicitações pessoais.

• Paciente aceita a versão do termo, recusa ou revoga solicitações.

• Médico consulta histórico autorizado, registra atendimento e consulta seus próprios registros.

hub-api.js centraliza Fetch e Bearer JWT. O token de acesso fica em sessionStorage por aba; ao vencer, o usuário entra novamente. Não se armazena senha nem refresh token no portal.

As listagens em outra aba exigem atualização manual. O visualizador revalida acesso a cada 30 segundos e a cada página; limpa a imagem ao fechar ou receber negativa. Não há atualização em tempo real das carteiras, aplicativo dedicado ou integração das telas legadas. Listas limitadas a 100 itens.

Organização do Código Frontend

frontend/hub.html e hub-login.html são templates; frontend/css, frontend/Js e frontend/img contêm os recursos. O portal deve ser servido pelo Django; a publicação estática legada não executa esse fluxo.

22.2 Implementação Backend

Views, serializers, permissions, services e selectors compõem backend/hub. A autoria do atendimento é definida pela conta autenticada, sem escolha de autor pelo cliente.

22.2 Implementação Backend (continuação)

Método / rota em /api/v1/	Regra principal
POST auth/login/; POST auth/refresh/	Autenticar e renovar token.
GET me/; GET doctor/patients/	Identidade e carteira permitida.
GET / POST doctor/grants/	Listar e solicitar acesso por médico.
GET patient/grants/; POST patient/grants/{id}/decision/	Titular decide; aceite exige versão do termo.
GET doctor/patients/{id}/history/	Grant pessoal vigente e recorte do aceite.
POST doctor/patients/{id}/records/	Criar atendimento com Grant vigente e contexto clínico validado.
GET doctor/patients/{id}/records/	Somente registros do próprio autor ativo.
POST integration/records/	Ingestão anterior: chave da integração e vínculo médico.

O histórico compartilhado inclui apenas registros ocorridos e recebidos até o aceite. Registros futuros exigem nova autorização. Permissão não se transfere a colegas e não depende de aceite adicional da clínica de origem.

O cadastro direto dispensa chave de integração. Rejeita data futura e descrição vazia. Clínica é opcional; quando informada, exige vínculo médico ativo. A criação e a auditoria ocorrem na mesma transação.

Após revogação ou expiração, sem outro Grant pessoal vigente, a leitura compartilhada e novos cadastros são bloqueados. A leitura por autoria permanece separada e não libera registros alheios. Não há edição ou exclusão de atendimentos pela API.

22.3 Integração Frontend e Backend

1. Protocolo e Segurança na Comunicação

Fetch envia HTTP com JSON, multipart e páginas PNG para /api/v1/ e Authorization: Bearer. O ambiente demonstrado usa HTTP local. JWT identifica o usuário; as permissões são verificadas no backend, sem roteamento de tenant. O navegador não acessa o banco diretamente.

2. Fluxos Principais do Sistema (Ponta a Ponta)

• Login: API valida credenciais, retorna token e perfil; o portal carrega a carteira ou as solicitações do paciente.

• Compartilhamento: médico solicita com UUID, finalidade e prazo; paciente lê e decide; a API grava e audita o Grant.

• Histórico: médico consulta; serviço verifica perfil, autorização e recorte temporal antes de retornar Record.

• Atendimento: formulário envia data, descrição e clínica opcional; serviço valida, grava Record/Audit e retorna 201.

• Revogação: decisão do paciente bloqueia novos acessos compartilhados sem outro Grant válido; registros próprios continuam na rota de autoria.

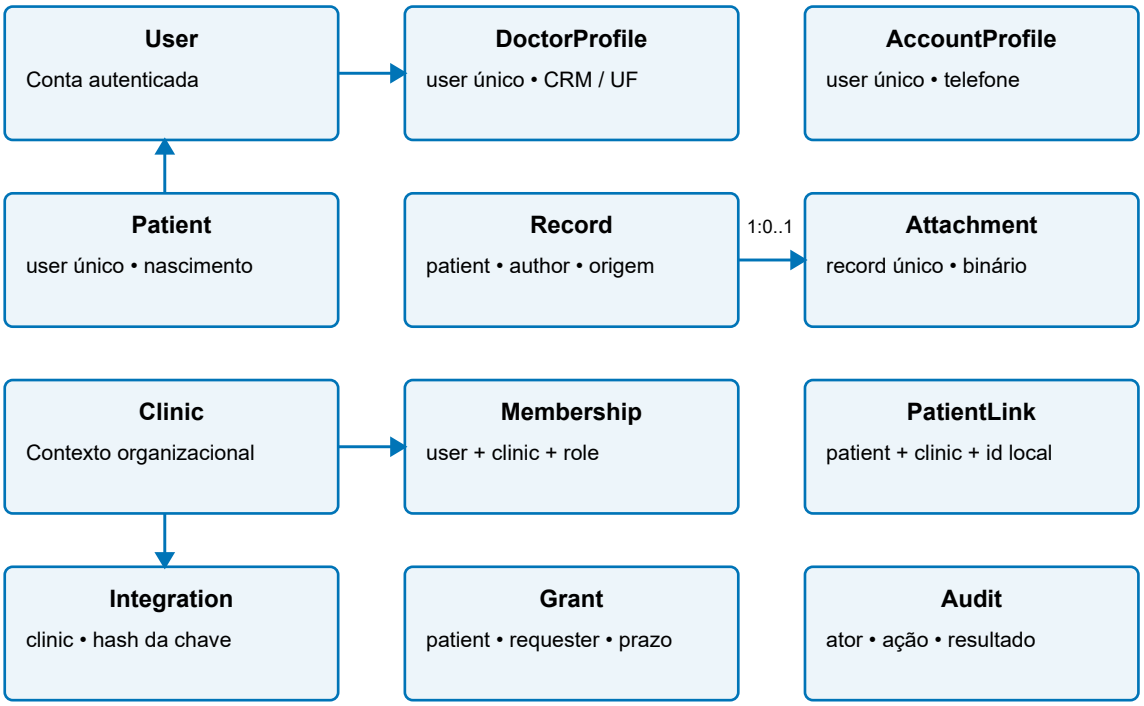
3. Resiliência e Integridade de Dados

Transações e bloqueios coordenam decisões e operações críticas. HTTP: 200/201 para sucesso; 400 entrada inválida; 401 autenticação; 403 permissão; 404 recurso ausente; 409 conflito. Ingestão repetida com mesmo conteúdo é idempotente; a rota de cadastro direto ainda não possui chave de idempotência.

Validação consolidada em 17/10/2026: 45 testes passaram no PostgreSQL. Migrações 0001 a 0006 aplicadas. Fluxos anteriores e visualizador foram conferidos no navegador; a inspeção visual dos novos perfis ficou bloqueada pela política da ferramenta. Revisão de chaves, correções de registros e implantação permanecem pendentes.

20 A 22 - COMPLEMENTO DO INCREMENTO 1.2

Versão consolidada em 17/10/2026. Pacientes e Autorizações usam visões separadas; Meu perfil existe para médico e paciente. Meu histórico oferece consulta integral ao titular e contribuição identificada.



DER resumido: FKs e regras adicionais descritas na seção 18.

Mesmo banco/schema; anexos privados vinculados ao registro.

AccountProfile (1:1 User) armazena telefone; DoctorProfile (1:1 User) inclui CRM/UF e especialidade. Attachment tem relação de 0..1 para cada Record. Conteúdo binário privado fica no mesmo PostgreSQL no protótipo. Não há endpoint público de mídia.

Migrações 0005 e 0006 adicionam anexos e perfis. Atualização de perfil é transacional, auditada e restrita à conta autenticada; não altera habilitação, destinatários ou termos anteriores.

22.2 E 22.3 - DOCUMENTOS, PERFIS E INTEGRAÇÃO

Endpoint / recurso	Regra implementada
POST doctor/grants/	duration_days: 10, 30, 60, 90; expires_at calculado no envio.
GET/POST patient/records/	Titular consulta e contribui; autoria do paciente identificada.
POST doctor/.../records/	Descrição e arquivo multipart; consentimento vigente.
GET attachments/{id}/preview/	Metadados privados para o visualizador.
GET attachments/{id}/pages/{n}/	PNG identificado, controle por página e sem arquivo original.
GET attachments/{id}/download/	Somente titular. Médico, inclusive autor, recebe HTTP 403.
GET/PATCH me/profile/	Contato, nome e dados específicos; sem alteração de papéis.

Pillow 12.3.0 e pypdfium2 5.14.0 rasterizam e recodificam os anexos. Um PDF/JPG/PNG por registro, até 10 MB; PDF de até 100 páginas, lado máximo de 1800 pixels. PDFium é serializado entre threads. Não há entrega do PDF original ao médico.

Controle por titularidade, autoria médica habilitada ou Grant que inclua o registro. Cabeçalhos no-store/nosniff e auditoria por consulta. Revalidação no visualizador a cada 30 segundos e a cada página. Não impede captura de tela nem cópia da imagem recebida.

45 testes aprovados no PostgreSQL em 17/10/2026. Visualizador conferido no navegador; perfis validados por testes, com inspeção visual bloqueada pela ferramenta. Scanner completo, validação física em celular e segurança de produção permanecem pendentes.